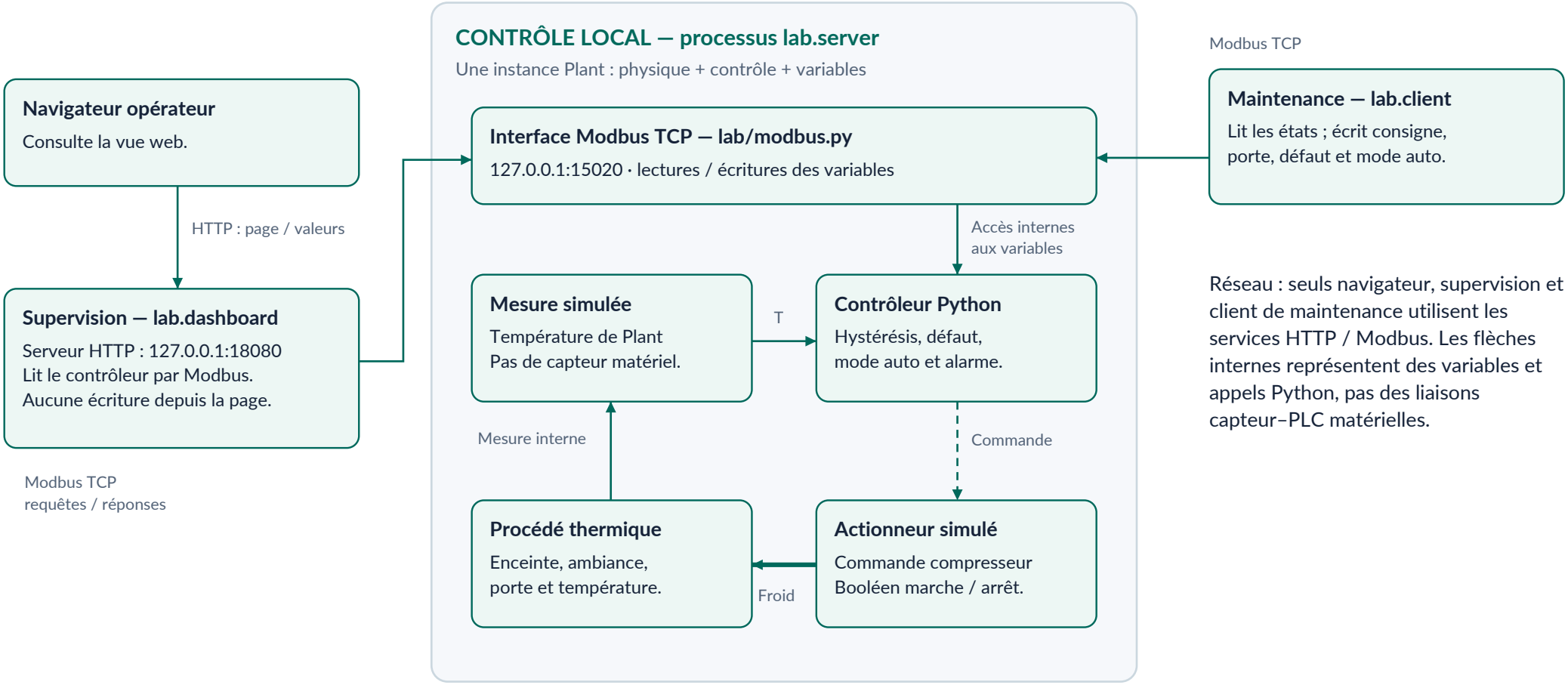
J1 — Schéma d'architecture OT annoté

Critère : identifier les composants, leurs fonctions et interactions ; distinguer contrôle local et supervision.

Architecture du laboratoire de référence Python/Modbus



Réseau : seuls navigateur, supervision et client de maintenance utilisent les services HTTP / Modbus. Les flèches internes représentent des variables et appels Python, pas des liaisons capteur–PLC matérielles.

Lecture : traits fins = données / échanges ; pointillés = commande ; trait épais = effet physique simulé. Les requêtes et réponses réseau empruntent la même liaison logique.

Périmètre : fonctions OT exécutées sur informatique Linux. Ce schéma n'est pas un plan de segmentation. OpenPLC, FUXA et la passerelle optionnelle ne sont pas intégrés au chemin démontré.

Composants, régulation et dépendances

Analyse du fonctionnement à partir des fichiers du laboratoire et des essais J1.

Composant / fonction	Rôle et interaction identifiés
Enceinte / processus	Plant.advance() calcule une température unique. L'ambiance réchauffe ; le compresseur refroidit. La porte ouverte double l'apport thermique du modèle.
Capteur simulé	Plant.temp est la mesure utilisée par le contrôle et publiée en registre. Un défaut injecté ne fait pas disparaître la température calculée.
Contrôleur local	Plant.advance() décide de la commande. À consigne 4 °C : marche ≥ 4,5 °C ; arrêt ≤ 3,5 °C ; conservation de l'état entre les seuils, en mode automatique.
Compresseur / actionneur	Plant.compressor active le terme de refroidissement. Il s'agit d'une commande simulée, sans retour matériel indépendant.
Alarme haute	Le modèle exige T > 8 °C pendant au moins 10 s calculées. Le compteur et l'alarme reviennent à zéro lorsque T ≤ 8 °C.
Supervision / IHM	lab.dashboard interroge Modbus et sert la page web : température, consigne, compresseur, porte et alarme. Le défaut capteur n'est pas affiché.
Maintenance / communications	lab.client lit les états et écrit des paramètres. Consigne : holding 0 (dixièmes de °C) ; température : input 0 ; compresseur : input 1 ; alarme : input 2 ; porte : coil 0. Adresses PDU à partir de zéro.

Contrôle local ≠ supervision

Arrêter seulement lab.dashboard : perte de la vue web. Si lab.server reste actif, sa boucle continue à calculer et commander. Cette autonomie est établie par le code ; aucun essai de coupure de supervision n'est capturé.

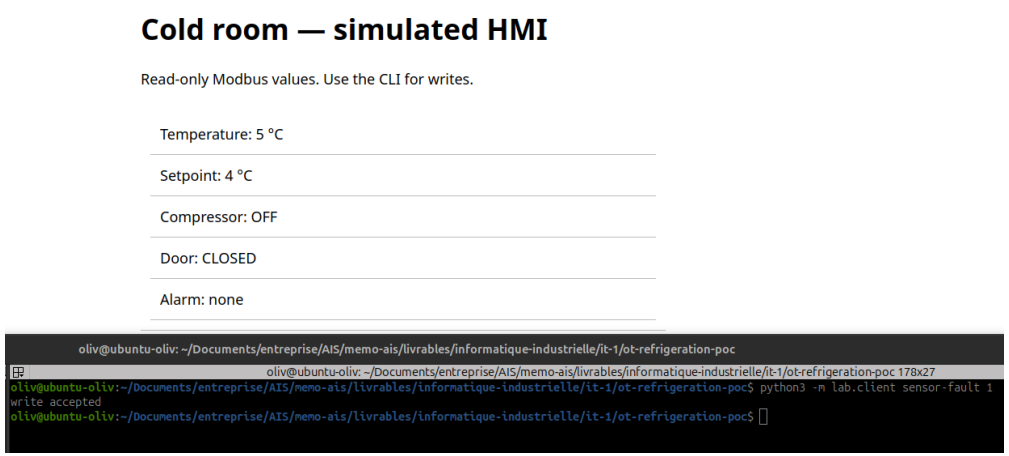
Redémarrer la machine qui héberge les deux : interruption du contrôle comme de la supervision. Relancer lab.server recrée les valeurs initiales ; la simulation arrêtée ne reproduit pas une enceinte réelle qui continue à se réchauffer.

Essais J1 : changement de consigne 4 → 6 → 4 °C, porte ouverte / fermée, défaut capteur et modification des seuils ±0,5 → ±1 °C observés en captures. La variante élargie est documentée ; sa restauration n'est pas encore attestée.

Preuves visuelles et portée des résultats

J1 — défaut capteur : distinguer l'affichage de supervision et la décision du contrôleur.

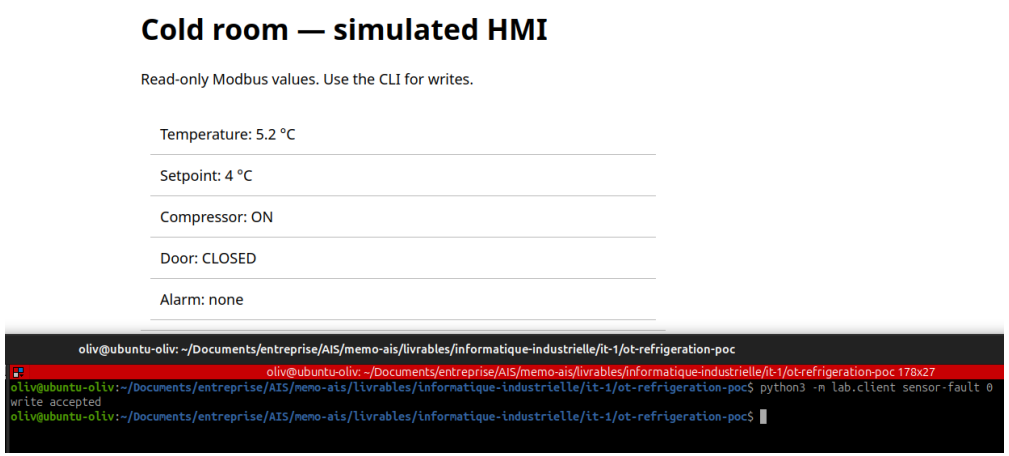
Défaut injecté : le compresseur reste arrêté



À 5 °C, pour une consigne de 4 °C, le compresseur est OFF : le seuil normal de 4,5 °C est dépassé. La commande sensor-fault 1 est acceptée. L'affichage ne montre pas le défaut et indique encore « Alarm: none ». La priorité au défaut est cohérente avec le code.

Capture utilisateur : 26/10/2026 11:25:05 (nom du fichier).

Défaut supprimé : la commande reprend



À 5,2 °C, après sensor-fault 0 accepté, le compresseur est ON. La température ne revient pas instantanément à la consigne. Les deux images montrent des états successifs ; elles ne mesurent pas précisément le délai de reprise.

Capture utilisateur : 26/10/2026 11:25:31 (nom du fichier).

Ce que le travail établit

Composants et interactions repérés ; régulation localisée dans le serveur Python ; supervision identifiée comme lectrice ; distinction entre consigne, commande et température ; réaction au défaut et reprise documentées.

Limites à conserver

Installation fictive, sans capteur ni automate matériel. Aucun OpenPLC ou FUXA connecté au procédé. Pas de validation de sûreté frigorifique, de conformité alimentaire ou de segmentation réseau. Le choix d'arrêt sur défaut est propre au scénario.

Sources de la preuve : laboratoire ot-refrigeration-poc : README.md, lab/model.py, lab/server.py, lab/modbus.py, lab/client.py, lab/dashboard.py et docs/REGISTER_MAP.md ; captures utilisateur J1. Le détail des essais est conservé dans les fiches 1.3 à 1.7 du mémo.